

CS-200 · Suivi des slides 2d : Exceptions

Oct 9, 2026 · @Gab

Ce document suit les slides 2d dans l'ordre et en explique chaque idée. Les slides 2a/2b ont présenté les interruptions d'entrée/sortie ; les slides 2d les replacent dans un cadre plus général : tous les cas où le flot de contrôle change sans que le programme l'ait écrit.

Quatre erreurs ou imprécisions des slides sont signalées au passage : la vectorisation RISC-V (slide 14), deux problèmes dans le handler de la slide 19, et la constante de `startADC` (slide 36). Il vaut la peine de les repérer avant l'examen.

1. Vocabulaire (slides 2–3)

Une exception est un changement de flot de contrôle que le programme ne contient pas explicitement. En temps normal, le programmeur décide seul de la suite des instructions exécutées, au moyen de `jal`, `beq`, `ret`, etc. Une exception est au contraire déclenchée par une condition particulière et transfère le contrôle à un **handler**.

Les noms varient beaucoup d'une architecture à l'autre (trap, fault, check...). Le cours suit la convention RISC-V / COD :

- **exception** désigne la catégorie générale ;
- **interruption** désigne le sous-ensemble des exceptions générées à l'extérieur du processeur.

Les interruptions d'I/O vues précédemment ne sont donc qu'un cas particulier.

2. Instruction indéfinie et émulation (slides 4–7)

Une instruction indéfinie lève une exception synchrone, ce qui permet de l'émuler en logiciel.

Le matériel. Le décodeur (le contrôleur qui lit `IR`) détecte qu'aucune instruction ne correspond à l'encodage et lève un signal `undef`. Ce signal est combiné par un OU avec `IRQ`. Le résultat commande le multiplexeur du PC suivant, qui sélectionne alors l'adresse du handler au lieu du PC calculé normalement. Le PC de retour est sauvegardé au même moment. On aurait pu simplement arrêter le processeur sur `undef`, mais sauter à un handler offre beaucoup plus de possibilités.

La différence avec une interruption I/O. Cette exception est **synchrone**. Formellement : si on relance le programme depuis le même état initial, l'exception se produit à chaque fois, et sur la même instruction. Une interruption I/O peut arriver n'importe où, selon le

moment où le périphérique termine.

Pourquoi c'est important. Si le matériel garantit que l'exception est servie *avant* l'instruction suivante, l'état du processeur est exactement celui qui précède l'instruction fautive. On peut alors **émuler** l'instruction en logiciel. C'est l'exemple de `fadd.s` : un processeur bon marché sans unité flottante lève une exception sur `fadd.s`, et le handler :

1. sauvegarde tous les registres qu'il va toucher. Les conventions d'appel ne s'appliquent pas ici, car le code interrompu n'a rien appelé et ne s'attend à aucune modification ;
2. lit l'instruction fautive en mémoire, à l'adresse sauvegardée ;
3. la décode en logiciel ;
4. lit les registres sources et calcule l'addition flottante (bibliothèque logicielle) ;
5. écrit le résultat dans le registre destination ;
6. avance le PC de 4 pour sauter l'instruction émulée ;
7. y retourne.

Le programme utilisateur ne voit aucune différence, à part la lenteur.

3. Pourquoi et comment classer (slides 8-10)

Les exceptions se classent selon trois axes, et certains choix relèvent d'une politique plutôt que du matériel.

Les causes possibles sont nombreuses : I/O, timer, instruction non supportée, faute arithmétique (division par zéro), violation de protection mémoire, débogage, panne matérielle, coupure de courant.

Les trois axes de la slide 9 :

- **synchrone ou asynchrone** : l'exception est-elle liée à une instruction précise ?
- **demandée ou imposée** (*coerced*) : le programme l'a-t-il provoquée volontairement (par exemple avec `ecall`) ?
- **reprise ou terminaison** : peut-on continuer l'exécution après le handler ?

Quelques lignes du tableau de la slide 10 méritent d'être comprises plutôt qu'apprises par cœur :

- **Page fault → resume.** L'OS charge la page manquante puis **ré-exécute la même instruction**. On reprend donc à `mepc`, pas à `mepc + 4`.
- **Misaligned access → resume.** Le handler peut émuler l'accès avec plusieurs `lb / sb` puis sauter l'instruction.

- **Undefined instruction → terminate.** Le tableau la classe ainsi, alors que l'exemple `fadd.s` montre qu'on *peut* la reprendre. Le tableau n'est qu'« une classification possible » : resume ou terminate relève souvent d'une **politique** de l'OS, pas d'une contrainte matérielle. C'est aussi le sens de l'exemple de la coupure de courant : éteindre ou hiberner.

4. Watchpoint (slide 11)

Un watchpoint lève une exception quand un registre donné reçoit une valeur donnée. On ajoute deux registres spéciaux, `Register` et `Value`, écrits par l'utilisateur. Des comparateurs observent le port d'écriture du banc de registres, et l'exception est levée si :

```
\text{wren} \land (\text{aw} = \text{Register}) \land (\text{wrdata} = \text{Value})
```

C'est l'outil d'un débogueur : « arrête-toi quand `x10` prend la valeur 42 ». L'exception est synchrone et demandée par l'utilisateur.

5. Les trois questions de conception (slides 12-14)

Dès qu'il existe plusieurs causes, il faut décider où sauver le PC de retour, comment identifier la cause et quoi faire d'une exception imbriquée.

Où sauver le PC de retour ? Pas dans `ra`, puisque le code interrompu l'utilise peut-être. Il faut un **EPC** (*Exception PC*) et une instruction de retour spéciale qui l'utilise. Trois solutions existent :

- un registre ordinaire réservé (Nios II) ;
- la pile (x86) ;
- des registres spéciaux avec leurs instructions d'accès (MIPS, RISC-V).

La valeur sauvegardée dépend du type d'exception :

- **asynchrone** : on sauve la prochaine instruction à exécuter ;
- **synchrone** : on sauve l'instruction fautive. On y revient pour réessayer après correction (page fault), ou on saute à la suivante si on a émulé (`fadd.s`).

Quelle est la cause ? La slide 14 présente trois schémas :

Schéma	Saut effectué	Contenu de la table
Handler unique + registre de cause	<code>pc ← constante</code>	aucune (dispatch logiciel)

Schéma	Saut effectué	Contenu de la table
Table d'adresses (68k)	$pc \leftarrow mem[EVA + 4 \cdot N]$	des adresses : saut indirect, un accès mémoire de plus
Table de code (SPARC, PA-RISC)	$pc \leftarrow EVA + 32 \cdot N$	du code : 32 octets = 8 instructions par entrée

Dans le troisième schéma, si le handler dépasse 8 instructions, on termine l'entrée par un saut vers la suite. C'est le `j FPU_Emulator_cont` de la figure.

Précision sur RISC-V. La slide range RISC-V dans le deuxième schéma (« table d'adresses »), ce qui n'est pas exact au regard de la spécification. En mode vectorisé (`mtvec.MODE = 1`), RISC-V fait $pc \leftarrow BASE + 4 \cdot cause$, sans lire la mémoire. Chaque entrée de 4 octets est donc une **instruction**, typiquement un `j handler_N` : c'est le troisième schéma avec des entrées de 4 octets. De plus, seules les interruptions sont vectorisées ; toutes les exceptions synchrones vont à `BASE`. En mode direct (`MODE = 0`), tout va à `BASE`, ce qui correspond au premier schéma, celui du cours.

Que faire si une exception survient pendant le handler ? On désactive ce qui peut l'être, c'est-à-dire les interruptions. On évite par construction ce qui ne peut pas être désactivé : le handler ne doit pas lui-même diviser par zéro ou exécuter une instruction indéfinie.

6. Les CSR de RISC-V (slides 15–18 et 20)

RISC-V gère les exceptions avec six CSR, deux séquences matérielles (entrée de trap et `mret`) et des instructions atomiques d'accès aux CSR.

Les registres (le sous-ensemble utilisé en CS-200) :

CSR	Bits utiles	Rôle
<code>mstatus</code>	MIE (bit 3), MPIE (bit 7)	autorisation globale des interruptions, et sa valeur avant le trap
<code>mie</code>	MTIE (bit 7), MEIE (bit 11)	autorisation par source
<code>mip</code>	MTIP (bit 7), MEIP (bit 11)	sources actuellement en attente
<code>mtvec</code>	BASE [31:2], MODE [1:0]	adresse du handler
<code>mepc</code>	tout	PC de l'instruction concernée
<code>mcause</code>	bit 31 = interruption, [30:0] = code	cause

Condition de prise d'une interruption externe en mode machine :

$$\text{take} = \text{mstatus.MIE} \ \&\& \ \text{mie.MEIE} \ \&\& \ \text{mip.MEIP}$$

`mip` reflète l'état du matériel : le périphérique lève `MEIP`. `mie` et `mstatus.MIE` sont deux niveaux de masque contrôlés par le logiciel, un par source et un global.

Ce que fait le matériel à l'entrée d'un trap :

```
mepc      ← pc          (instruction fautive, ou première non exécutée)
mcause    ← {interrupt, code}
mstatus.MPIE ← mstatus.MIE
mstatus.MIE ← 0          (masquage des interruptions)
pc        ← mtvec.BASE   (en mode direct)
```

Et à `mret` (slide 15) :

```
mstatus.MIE ← mstatus.MPIE
mstatus.MPIE ← 1
pc          ← mepc
```

La slide 17 précise que `mepc` pointe sur une instruction **non exécutée**. Il en découle une règle simple :

- **interruption** : on fait `mret` sans toucher `mepc`, et l'instruction interrompue s'exécute enfin ;
- **exception émulée** : on fait `mepc ← mepc + 4` avant `mret` ;
- **exception à réessayer** (page fault) : on ne touche pas `mepc`.

Les instructions CSR. Elles sont atomiques : lecture de l'ancienne valeur dans `rd` et modification en une seule instruction. Les pseudo-instructions de la slide 16 sont des cas particuliers avec `x0`. Par exemple, `csrr rd, csr = csrrs rd, csr, x0`, et un OU avec 0 ne modifie rien.

Exemple concret pour activer les interruptions externes :

```
la    t0, handler
csrw  mtvec, t0          # MODE = 0 (direct), car handler est aligné sur 4
li    t0, 0x800          # 1 << 11 = MEIE
csrs  mie, t0
csrsi mstatus, 0x8       # 1 << 3 = MIE
```

Pour `mie`, on ne peut pas utiliser `csrsi` : l'immédiat de ces instructions (`uimm`) fait 5 bits, donc il vaut au plus 31, alors que $2^{11} = 2048$. Pour `mstatus.MIE`, $2^3 = 8 \leq 31$, donc `csrsi` suffit.

Le bit 31 de `mcause` (slide 18). En complément à deux, bit 31 = 1 si et seulement si la valeur est négative. D'où les tests `bltz` / `bgez` utilisés pour distinguer interruption et exception.

7. Le handler d'instruction indéfinie (slide 19)

Le handler range les registres comme un tableau indexé par numéro, ce qui rend l'émulation simple, mais le code de la slide contient deux erreurs.

Organisation de la pile. Le handler réserve 128 octets, soit 32 mots, et range `x_i` à l'adresse `sp + 4i`. Il sauve même `x0` (inutile en soi) et laisse un emplacement à `8(sp)` pour `x2`. La raison : le décodeur renvoie des **numéros** de registres (`rs1`, `rs2`, `rd`), et lire « le registre d'origine numéro `r` » devient un simple accès indexé :

```
slli t1, a1, 2      # t1 = 4·r
add  t1, sp, t1     # t1 = sp + 4·r
lw   a1, 0(t1)      # a1 = valeur d'origine de x_r
```

L'écriture du résultat dans `rd` suit le même principe avec `sw`. Comme les registres sont restaurés depuis ce tableau à la fin, la modification devient effective au retour.

Le dispatch.

- `csrr a1, mcause` puis `bltz` : si le bit 31 vaut 1, c'est une interruption.
- Sinon, on compare à 2 (*illegal instruction*).
- `csrr t0, mepc` puis `lw a0, 0(t0)` : on lit l'encodage de l'instruction fautive.

Première erreur : `t0` est corrompu à la sortie. Le handler restaure tous les registres, `t0` compris, puis exécute `csrr t0, mepc` / `addi` / `csrw`. La valeur d'origine de `t0` est écrasée, et le programme reprend avec un `t0` faux. Il faut ajuster `mepc` avant la restauration :

```
csrr t0, mepc
addi t0, t0, 4
csrw mepc, t0      # d'abord mettre mepc à jour...
lw   x1, 4(sp)     # ...puis restaurer (t0 = x5 inclus)
...
lw   x31, 124(sp)
addi sp, sp, 128
mret
```

Deuxième erreur : l'emplacement de `x2` n'est jamais écrit. Si l'instruction émulée utilise `sp` comme source, elle lit une valeur indéterminée. Pour être correct, il faudrait y stocker la valeur d'origine, c'est-à-dire `sp + 128`. C'est un détail, mais il illustre bien la slide 29 : écrire un handler correct est difficile.

8. La pile et les handlers (slides 21-24)

Un handler a besoin d'une pile sûre, que RISC-V fournit via `mscratch`, et aucun code ne doit stocker de données utiles sous `sp`.

Débordement de pile (slides 21-22). On ajoute un comparateur : si l'instruction écrit `x2` et que la nouvelle valeur est inférieure à `Limit`, une exception `watch` est levée.

Formellement :

$$\text{\texttt{\textit{wren}}} \wedge (\text{\texttt{\textit{aw}}} = 2) \wedge (\text{\texttt{\textit{wrdata}}} < \text{\texttt{\textit{Limit}}})$$

« We will do better » annonce la mémoire virtuelle (pages de garde) plus tard dans le cours.

Le paradoxe (slide 23). Si l'exception vient d'un débordement de pile, le handler ne peut pas utiliser la pile. Plus généralement, un handler ne peut toucher aucun registre avant de l'avoir sauvé, mais il lui faut un registre pour calculer où sauver. RISC-V résout ce problème avec `mscratch`, un CSR libre d'un mot. L'idiome classique consiste à y garder un pointeur vers une pile sûre, puis à l'échanger avec `sp` en une seule instruction atomique :

```
handler:
    csrrw sp, mscratch, sp    # sp ← pile sûre ; mscratch ← sp utilisateur
    addi  sp, sp, -128
    sw    x1, 4(sp)
    ...                                     # traitement
    lw    x1, 4(sp)
    addi  sp, sp, 128
    csrrw sp, mscratch, sp    # on rétablit sp utilisateur, mscratch ← pile
sûre
    mret
```

Aucun registre n'est modifié avant d'être sauvé. Les handlers des slides 19 et 38 utilisent directement la pile du programme interrompu, ce qui fonctionne tant que cette pile est saine.

Les deux prologues de la slide 24 sont-ils équivalents ? Non, pas en présence d'interruptions. Dans la version de droite, on écrit sous `sp` (à `-12(sp)` ...) avant de décrémenter `sp`. Entre le premier `sw` et `addi sp, sp, -12`, ces données se trouvent dans une zone que la convention considère comme **libre**. Si une interruption survient à ce moment, le handler exécute `addi sp, sp, -120` puis ses `sw`, et écrase ces données. L'épilogue a le même problème : après `addi sp, sp, 12`, les `lw -12(sp)` lisent une zone déjà libérée.

La règle est donc : **ne jamais stocker de données utiles sous `sp`** . Certains ABI garantissent une *red zone*, par exemple les 128 octets sous `rsp` en x86-64 System V, que les handlers s'engagent à ne pas toucher. L'ABI RISC-V n'en a pas, d'où la version de gauche.

9. Protection et modes de privilège (slides 25–27)

Les modes de privilège réservent le matériel à l'OS, et les exceptions servent de passage contrôlé entre modes.

Slide 25. Un comparateur détecte qu'un accès mémoire vise l'adresse d'un périphérique (ici `0x10000000`). Si le bit de mode vaut « user », une exception est levée. Seul l'OS peut parler au matériel.

Slide 26. Pour que ce mécanisme ait un sens, il faut trois choses :

1. au moins deux modes. RISC-V en a trois : Machine, Supervisor, User ;
2. un état lisible par tous mais modifiable seulement en mode privilégié, sinon l'utilisateur repasserait simplement en mode superviseur ;
3. un moyen contrôlé de changer de mode. `ecall` provoque volontairement une exception, qui fait entrer dans le mode privilégié à **une adresse choisie par l'OS** (`mtvec`) et non par l'utilisateur. `mret` / `sret` font le chemin inverse.

Slide 27. Les tâches du processeur lors d'une exception correspondent ainsi aux mécanismes RISC-V :

Tâche	Mécanisme RISC-V
Masquer les interruptions	<code>MIE ← 0</code>
Sauver l'EPC	<code>mepc</code>
Sauver la cause	<code>mcause</code>
Élever le privilège	passage en mode M ; l'ancien mode est sauvé dans <code>mstatus.MPP</code> , hors du sous-ensemble du cours
Libérer des registres	pas de registres fantômes en RISC-V, d'où <code>mscratch</code>
Sauter au handler	<code>pc ← mtvec</code>

`mret` défait implicitement le masquage et le changement de privilège. En revanche, réactiver les interruptions *au milieu* du handler reste une décision du programmeur.

10. Priorités, imbrication, exceptions précises (slides 28–30)

Autoriser une interruption prioritaire pendant un handler exige de sauver les CSR et de re-masquer avant `mret`.

Imbrication. Il n'existe qu'un seul `mepc` et un seul `mcause`. Si une deuxième interruption est prise pendant le handler, ils sont écrasés et le retour vers le code d'origine est perdu. C'est pourquoi le matériel met `MIE ← 0` automatiquement. Pour permettre à une interruption prioritaire de préempter le handler, la séquence correcte est :

1. sauver `mepc`, `mcause` et `mstatus` sur une pile sûre ;
2. réactiver les interruptions avec `csrsi mstatus, 8` ;
3. faire le traitement long ;
4. **re-masquer** avec `csrci mstatus, 8` ;
5. restaurer `mepc` et `mstatus` ;
6. `mret`.

L'étape 4 est souvent oubliée. Sans elle, une interruption qui arrive entre la restauration de `mepc` et `mret` écrase à nouveau `mepc`.

Slide 29. Elle résume les pièges : pile éventuellement inutilisable, handler **non réentrant** s'il utilise des emplacements statiques (comme `mscratch`), latence maximale à respecter pour ne pas perdre de données d'I/O.

Slide 30. Une exception est **précise** si, au moment où le handler démarre :

- toutes les instructions avant `mepc` ont terminé ;
- aucune instruction à partir de `mepc` n'a modifié l'état.

C'est cette propriété qui rend l'émulation et la reprise possibles. Sur un processeur séquentiel simple, elle est gratuite. Sur un processeur pipeliné, où plusieurs instructions sont en vol simultanément, elle devient difficile à garantir ; on y reviendra avec le pipeline.

11. L'exemple du convertisseur A/D (slides 31–41)

L'interruption remplace la boucle d'attente active ; le code de l'exemple est juste, sauf la constante de `startADC`.

Le problème (slide 31). La boucle `poll: lw / beqz` occupe le processeur à 100 % pendant toute la durée de la conversion, sans rien produire d'utile.

La nouvelle interface (slide 32).

- Tout accès à `0xFFFFF0` démarre une conversion.
- La fin de conversion lève une interruption.

- Le résultat se lit à `0xFFFFF8`.

La poignée de main IREQ/IACK (slide 33). IREQ est un signal de **niveau** : le périphérique le lève quand il veut (asynchrone) et le **maintient jusqu'à l'acquittement**. Le processeur répond par IACK quand il prend l'interruption, ce qui fait retomber IREQ. Le maintien garantit que la requête n'est pas perdue si le processeur a momentanément masqué les interruptions.

Le circuit (slide 35). La sortie `/DV` est inversée pour donner `DV`. Un front montant de `DV` signifie « conversion terminée ». Ce front sert d'horloge à une bascule D dont l'entrée D est câblée à 1 : la bascule passe donc à `Q = 1`, et `Q` pilote IREQ3. IACK3 est relié au reset de la bascule. La bascule transforme un **événement** (un front, potentiellement bref) en un **état** maintenu jusqu'à l'acquittement, ce qui réalise exactement le chronogramme de la slide 33. Sur la slide 34, « IACQ3 » est une coquille pour IACK3.

startADC (slide 36) : la constante est fausse. La séquence `lui t0, 0xffff` puis `addi t0, t0, 0xff0` ne donne pas `0xFFFFF0` :

- `lui t0, 0xffff` donne `t0 = 0xFFF << 12 = 0x00FFF000` ;
- l'immédiat de `addi` est un entier **signé sur 12 bits**. `0xff0` a son bit 11 à 1, donc il vaut $0xFF0 - 2^{12} = -16$. GNU `as` le refuse d'ailleurs, car il sort de l'intervalle $[-2048, 2047]$;
- résultat : $0xFFFF000 - 16 = 0xFFEFF0$, et non `0xFFFFF0`.

Pour trouver la bonne décomposition, on cherche `hi` et `lo` tels que `valeur = (hi << 12) + sext(lo)`. Les 12 bits de poids faible de `0xFFFFF0` sont `0xFF0`, et `sext(0xFF0) = -16`. Il faut donc :

$$\text{hi} = \frac{\text{0xFFFFF0} + 16}{2^{12}} = \frac{\text{0x1000000}}{2^{12}} = \text{0x1000}$$

```
lui  t0, 0x1000    # t0 = 0x01000000
addi t0, t0, -16    # t0 = 0x00FFFFFF0
```

C'est exactement ce que génère `li t0, 0xfffff0`, utilisé correctement dans `readADC` (slide 39). Ce piège de la retenue entre `lui` et `addi` revient souvent à l'examen.

Le handler (slide 38).

- Il sauve 30 registres (`x1` et `x3 ... x31`), soit 120 octets.
- `bgez s0` : si le bit 31 vaut 0, ce n'est pas une interruption.
- `slli s0, s0, 1` puis `srls s0, s0, 1` effacent le bit 31. Le décalage à gauche éjecte ce bit, et le décalage logique à droite réinsère un 0. Algébriquement, on calcule `s0 mod 231`, c'est-à-dire le code seul. Alternative en une comparaison : `li s1, 0x8000000B` puis `bne s0, s1, ...`.

- Si le code vaut 11 (*machine external interrupt*), on lit l'ADC et on stocke la valeur.
- **mepc n'est pas modifié**, et c'est voulu. C'est une interruption : **mepc** désigne l'instruction interrompue, qui n'a pas encore été exécutée et doit l'être au retour. C'est la différence avec la slide 19.

readADC (slide 39). $0xFFFFF0 + 8 = 0xFFFFF8$: le décalage de 8 dans `lw a0, 8(t0)` désigne le registre de donnée.

insertIntoBuffer (slide 40). C'est un tampon circulaire. L'astuce est la mise à jour de l'indice $i \leftarrow (i+1) \& (N-1)$. Si $N = 2^k$, alors $N - 1$ s'écrit avec k bits à 1, et le ET garde exactement les k bits de poids faible. On obtient donc $(i+1) \bmod 2^k$. Cette égalité n'est vraie que pour une puissance de deux, d'où le commentaire « must be a power of two ».

Deux limites de cet exemple :

- il n'y a pas de pointeur de lecture : si le programme principal ne consomme pas assez vite, les anciennes valeurs sont écrasées silencieusement ;
- **bufferPointer** est une variable statique, ce qui n'est acceptable ici que parce que les interruptions sont masquées pendant le handler. Le programme principal qui lit le tampon devra gérer les accès concurrents.

Synthèse

Le cours se résume en trois idées :

1. **Le matériel en fait le minimum.** Il sauve où il était (**mepc**) et pourquoi (**mcause**), masque les interruptions et saute à **mtvec**. Tout le reste est du logiciel.
2. **Le handler doit être invisible.** Il sauve tout ce qu'il touche, sans compter sur les conventions d'appel, et retourne à **mepc** (interruption ou réessai) ou à **mepc + 4** (émulation).
3. **Les vraies difficultés sont aux frontières.** Il faut une pile sûre (**mscratch**), éviter les données sous **sp**, gérer l'imbrication (sauver les CSR, re-masquer avant **mret**) et garantir des exceptions précises dans un processeur pipeliné.

Pour réviser, le plus rentable est de savoir refaire de tête la séquence d'entrée d'un trap, celle de **mret**, et la règle « **mepc** ou **mepc + 4** ».